

AI Meeting Assistant

From Meeting Audio to Structured, Actionable Records

INTER IIT BOOTCAMP

MULTIMODEL AI PIPELINE

Problem

Recorded meetings contain decisions, technical terminology, commitments, deadlines and assignments, but raw audio is difficult to review and operationalize.

Solution



Three Inspectable Outputs

Raw Transcript What the STT model heard; preserved before any LLM transformation.

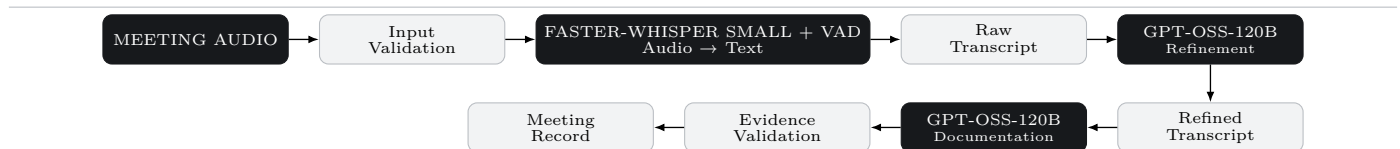
Refined Transcript Domain-aware correction of terminology, names, numbers and grammar while preserving meaning.

Final Record Summary, minutes, confirmed decisions, action items, owners, deadlines and evidence.

Design principle

Each model has one clearly defined responsibility, keeping transcription, refinement and documentation independently inspectable.

Architecture at a Glance



Why preserve raw?

If the final record is wrong, intermediate representations make it possible to determine whether the issue originated in audio, STT, refinement, documentation or evidence validation.

System contract

Raw = what was heard **Refined** = what was improved **Final** = what is supported.

Input contract

Recorded English audio is validated before inference. Empty, corrupt, unreadable or unsupported inputs are rejected explicitly.

Output contract

Human-readable and machine-readable artifacts are exported without silently replacing earlier valid representations.

Core design philosophy

Raw transcript answers *what the speech recognizer heard*; refinement answers *how the conversation can be represented more accurately*; documentation answers *what useful information can safely be extracted*; evidence validation checks the claims before export.

PRESERVE FIRST · IMPROVE SECOND · EXTRACT THIRD

Inspectable

Raw, refined and final representations remain independently visible.

Grounded

Decisions and tasks require transcript support.

Actionable

The final record is exportable in human and machine-readable forms.

Architecture rationale Preserve the raw conversation, improve it in a dedicated refinement stage, extract structured information separately, then validate claims against evidence. This makes the system inspectable rather than opaque.

Evaluator lens The architecture makes it possible to inspect three separate questions: What was heard? What was corrected? What was actually supported by the meeting? This separation is the central trust mechanism of the application.

KEY TAKEAWAY The architecture is intentionally a sequence of inspectable semantic transformations rather than one opaque LLM call.

Stage 1 — Speech Recognition

Turning meeting audio into an inspectable timestamped transcript.

faster-whisper small + Silero VAD

Audio → Text

Input: recorded English meeting audio

Output: timestamped transcript segments

The STT stage is deliberately isolated from the Groq language-model stages. It creates the first stable textual representation of the conversation.

Why local speech recognition?

- Local inference keeps the STT layer independent of an external transcription API.
- No separate hosted STT quota is required during inference.
- Segment timing remains available to downstream processing and inspection.
- The same raw artifact can be reused for debugging and evaluation.

STT Data Flow



What is preserved?

Segment ordering, transcript text and timing are retained before any refinement. The raw artifact is not silently replaced by later processing.

Why it matters

A final extraction error can be traced back to the speech-recognition output rather than being hidden inside a single final answer.

Validation Before Inference

File validation

Empty, corrupt, unreadable or unsupported inputs are rejected explicitly.

Audio readability

The pipeline checks that the recording can be processed before invoking downstream models.

Raw artifact

A successful STT run produces a stable transcript artifact for later stages.

Why the Raw Transcript Is a First-Class Output

The raw transcript is not merely an intermediate variable. It is an explicit product of the system. This creates an auditable boundary between **what the speech recognizer produced** and **what later language models changed**. If the refined transcript or final record contains an error, the evaluator can compare representations instead of trusting a single generated result.

Local STT boundary

Audio stays in the speech-recognition layer.

Text becomes the contract passed to LLM stages.

Groq is not responsible for transcription.

Inspectable artifact

`raw_transcript.txt` records the STT result independently from refinement and documentation outputs.

ENGINEERING NOTE Preserving the STT artifact prevents downstream LLM fluency from masking speech-recognition errors.

Timestamped segments

Timing remains available at the STT boundary, making the transcript inspectable beyond plain text.

Downstream contract

Later models receive text, not audio, so speech recognition and language reasoning can be evaluated independently.

What STT deliberately does not do

It does not summarize, infer decisions, identify owners, correct domain terminology through an LLM, or generate meeting minutes. Those operations begin only after a raw transcript exists.

Stage contract AUDIO → TIMESTAMPED SEGMENTS → RAW TRANSCRIPT. NO SUMMARIZATION. NO SEMANTIC REWRITING.

Stage 2 — Domain-Aware Refinement

Improve recognition quality without changing what was actually said.

openai/gpt-oss-120b

INFERENCE VIA GROQ

The refinement model receives the raw transcript and performs controlled language cleanup. Its responsibility is to improve transcript quality, terminology and readability while preserving the underlying meaning.

What the Refinement Stage Handles

Technical terms

Domain-specific vocabulary and plausible STT corrections.

Names + acronyms

Important entities should not be silently replaced by common alternatives.

Numbers

Numerical and monetary values must remain accurate.

Grammar

Improve readability and terminology consistency without changing meaning.

Protected Semantics

Names

Numbers

Negation

Commitments

Ordering

Meaning

Technical terms

Domain context

Number safety

“2.5 lakh” must not become “25 lakh”. Monetary values are treated as protected information.

Negation safety

“We will not deploy this version” must retain its negative commitment.

Commitment safety

“We will complete this tomorrow” must preserve its commitment semantics.

Optional Glossary Enrichment

Glossary can contain

technical terms · product names · project names · domain vocabulary · important names · acronyms

Failure policy

Glossary extraction is optional. If it fails, the system warns and continues refinement using fallback terminology rather than terminating the mandatory pipeline.

Long-Meeting Strategy

Long transcript → Token-aware chunking → Sequential GPT-OSS-120B processing → Ordered reconstruction → Refined transcript

Why chunk?

Large transcripts can exceed the practical token budget of one structured request. Chunking avoids relying on a fixed meeting-duration assumption.

Why sequential?

Chunks are processed in order and reconstructed in their original sequence so the refined transcript remains coherent.

What this stage does NOT do

It does not summarize the meeting, decide what counts as an assignment, invent missing owners, or convert proposals into decisions. Those responsibilities belong to the documentation and evidence stages.

Reliability Controls

Structured output

Explicit schemas, prompt alignment, validation and retry/backoff protect downstream stages from malformed responses.

Audit + fallback

Glossary failure is non-fatal; the refined transcript remains a separate artifact for raw-versus-refined inspection.

Stage contract RAW TRANSCRIPT → CONTROLLED REFINEMENT → REFINED TRANSCRIPT.
Preserve names, numbers, negation, commitments, ordering and meaning; do not summarize or invent.

KEY TAKEAWAY Refinement improves the transcript; it is not permission to rewrite the meeting.

Stage 3 — Documentation + Evidence

Turn the refined conversation into a structured record without inventing commitments.

openai/gpt-oss-120b

INFERENCE VIA GROQ

The documentation model receives the refined transcript and asks: *What useful meeting information can safely be extracted from this conversation?*

Structured Meeting Record

Summary

Concise representation of what happened during the meeting.

Decisions

Only decisions supported by the conversation.

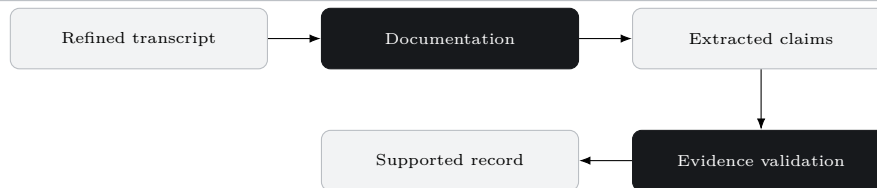
Action Items

Tasks supported by the conversation.

Owners / Deadlines

Included only when explicitly stated.

Evidence-First Extraction



Proposal
≠ Decision

Suggestion
≠ Assignment

Mentioned date
≠ Deadline

Possibility
≠ Commitment

Examples of Safe Extraction

Discussion

“We could move this to next week.” is a proposal, not a confirmed decision.

Assignment

“Maybe Jatin can handle it.” is a suggestion unless the meeting establishes an actual assignment.

Missing metadata

If no owner is stated, owner remains unspecified. If no deadline is stated, deadline remains unspecified.

Evidence boundary

A fluent LLM statement is not sufficient evidence. Extracted claims are checked against transcript evidence before entering the final record.

Human + Machine Outputs

Human-readable

minutes.md
meeting_record.md
Summary, decisions, action items and supporting context in an evaluator-friendly format.

Machine-readable

decisions.json
action_items.json
meeting_record.json
refinement_audit.json

Why two language models?

GPT-OSS-120B answers: *How can the transcript be represented more accurately without changing its meaning?*

GPT-OSS-120B answers: *What decisions and actionable information can safely be extracted?*

The separation makes the source conversation and the extracted meeting record independently inspectable.

Final Record Contract

Summary

What happened, expressed concisely.

Decisions + tasks

Only conversation-supported decisions and action items.

Evidence

Important extracted claims remain tied to transcript support.

Output guarantees The final record distinguishes confirmed information from missing metadata. Unsupported owners, deadlines, decisions or assignments are not filled in merely to make the output look complete.

Reliability principle The system prioritizes evidence over fluency. It should fail explicitly rather than silently invent information.

Reliability + Engineering

Hardening the pipeline around the failure modes exposed by real testing.

01 — Long Meeting Token Limits

Long transcripts can exceed the practical generation budget of one request. The final system uses token-aware chunking, sequential processing and ordered reconstruction rather than repeatedly retrying an oversized request.

02 — Structured JSON Reliability

Structured-output validation can fail when generated JSON is malformed. The pipeline uses strict schema alignment, synchronized prompts, sanitization, validation, retry/back-off and JSON-boundary recovery.

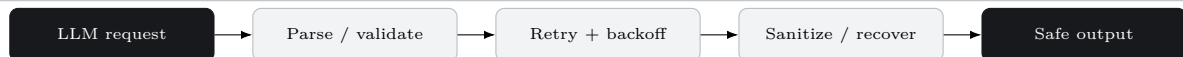
03 — Meaning Preservation

Regression checks protect negation, monetary values, numerical values, technical terminology, proposal-vs-decision distinctions and unspecified owners/deadlines.

04 — Stage Observability

The pipeline exposes explicit states: received, validated, transcribing, raw_ready, refining, refined, documenting, validating, exporting and complete.

Retry + Fallback Architecture



Failure Handling by Criticality

Component	Failure behavior	Engineering rationale
Optional glossary	Warning → continue	Enrichment must never bring down the mandatory pipeline.
Refinement	Explicit error → safe stop	Required semantic transformation cannot be silently skipped.
Documentation	Explicit error → safe stop	Final structured record must not be fabricated.
Evidence validation	Reject unsupported claim	Plausible language is not evidence.

Reasoning-token control

GPT-OSS models are reasoning models. Long structured requests can consume generation budget before valid JSON appears, so reasoning behavior and output budgets are controlled.

API-key protection

Exceptions are sanitized before logging so live credentials do not accidentally appear in terminal output or application error messages.

Long-Recording Verification

A real approximately 15-minute meeting exposed the practical token-budget problem. After token-aware chunking and the hardened request path, the full pipeline completed successfully.			
Raw transcript	Refined transcript	Decisions	Action items
14,073 characters	14,104 characters	2 confirmed	4 extracted

Stage-Level Observability

received → validated → transcribing → raw_ready → refining → refined → documenting → validating → exporting → complete

Debuggable

A failure can be localized to a stage instead of reported as a generic processing error.

Resilient

Optional enrichment degrades gracefully while required stages fail explicitly.

Traceable

Intermediate artifacts and stage states remain visible for evaluator review.

Engineering lessons Observable stages, graceful optional fallbacks, explicit token budgets, structured-output recovery and evidence validation turn real failure modes into testable engineering contracts.

Engineering takeaway Token limits, malformed structured output, API failures and semantic drift are treated as engineering constraints—not hidden behind a final generated paragraph.

UI + Testing + Final Status

A compact evaluator surface backed by automated tests and real end-to-end verification.

99 / 99

automated tests passing

Human-Centered Interface

Raw Transcript | Refined | Final Output

- **Raw Transcript** — original timestamped faster-whisper small + VAD output.
- **Refined Transcript** — domain-corrected transcript with preserved meaning.
- **Final Output** — summary, minutes, decisions, action items and evidence.

What the Interface Exposes

Raw view — unchanged STT artifact and timing.
Refined view — corrected terminology without rewriting the conversation.
Final view — evidence-grounded decisions and action items.
Download layer — intermediate and final artifacts remain independently exportable.

Coverage

Evidence · Pipeline · Refinement · Safety · UI

Real-meeting verification

A real approximately 15-minute recording completed audio → transcription → refinement → documentation → validation → export.

Evaluator path

1. Upload a new recording.
2. Compare raw vs refined text.
3. Inspect evidence-backed decisions.
4. Export the complete bundle.

Test Suite

Area	Verified
Pipeline	Audio validation, transcription flow, outputs and packaging.
Refinement	Chunking, ordering, edit safety, glossary fallback, JSON, retries and sanitization.
Safety	Negation, monetary values and safety-sensitive transformations.
Evidence	Decision classification, evidence matching and quote extraction.
UI	Status streaming, errors and three-stage output layout.

Downloadable Output Contract

```
raw_transcript.txt
refined_transcript.txt
minutes.md
decisions.json
action_items.json
meeting_record.json
meeting_record.md
refinement_audit.json
```

Technology Stack

Layer	Technology	Role
Language	Python	Pipeline implementation
Speech-to-text	faster-whisper small + Silero VAD	Audio → raw transcript
Refinement	GPT-OSS-120B	Domain-aware correction
Documentation	GPT-OSS-120B	Structured extraction
Inference	Groq	Hosted LLM inference
UI	Gradio	Upload, status, inspection, export
Output	JSON / Schema	Validated machine-readable record
Testing	Pytest	Regression suite

PS / Requirement Coverage

Requirement	Implementation	Status
English meeting audio	faster-whisper small + Silero VAD	YES
Raw transcript	Dedicated preserved artifact	YES
Domain refinement	GPT-OSS-120B	YES
Separate documentation stage	GPT-OSS-120B	YES
Minutes / summary	Structured meeting record	YES
Decisions	Evidence-aware extraction	YES
Tasks	Grounded action items	YES
Missing owner / deadline	Left unspecified	YES
Interactive UI	Gradio	YES
Downloads	TXT / MD / JSON	YES
New recordings	Dynamic processing pipeline	YES

UI reliability

Clear stage-level errors, readable transcript text, explicit light-theme styling and separate raw / refined / final views.

Evidence boundary

Unsupported owners, deadlines or decisions are not invented. Proposal, suggestion and mention remain distinct from commitment.

Current boundary

The submitted system processes recorded English meeting audio through the validated multimodel pipeline. Unsupported integrations and future features are outside the current architecture.

Final verification Upload → Transcribe → Refine → Document → Validate → Export.

The system preserves what was said first, then improves and structures it without inventing unsupported meeting facts.